QueryAPIGate DOCUMENTATION · PERFORMANCE

Performance, v5: a Rust data plane in front of /q/

Current as of QueryAPIGate **v0.49.0** · PDF made 4 November 2026

Latest version: anantharajuc.github.io/QueryAPIGate-website/Guide.dc.html?d=PERFORMANCE_V5

The question is ADR 0028's: does a Rust data plane, applying the plans Python computes, serve saved-query calls at least 3 times as fast per gateway core, with p99 no worse and the differential gate clean? **The case** is v4's on PostgreSQL (v4) and the TPC-C estate on MySQL (MySQL v2), with the data plane added in front.

The short answer: yes, on every lookup, on both databases, with every run recorded.

	PYTHON	PYTHON + CACHE	DATA PLANE	DATA PLANE + CACHE
PostgreSQL lookup, 32 callers	873/s	1,043	12,689	20,000 at 2.97 cores*
per gateway core	747	925	2,361 (**3.2x**)	6,734 (**7.3x**)
p99	42.8 ms	35.9 ms	5.1 ms	1.8 ms
MySQL customer lookup, 32 callers	794/s	898	15,871	23,961
per gateway core	616	769	2,144 (**3.5x**)	3,253 (**4.2x**)
p99	47.0 ms	39.7 ms	3.2 ms	2.1 ms
Both at their best on 8 cores, 64 callers (MySQL)		2,976/s, 8 workers		23,619/s, **7.9x per core**

* At a fixed 20,000 calls a second; flat out, the data plane + cache served 27,500 calls a second on PostgreSQL, but only about half of their runs were recorded (below). The multiples compare each data-plane build with the Python build behind it, cache with cache.

ADR 0028'S CRITERION	RESULT
The differential gate: the same answer as Python, call for call	**Clean**: 0 response and 0 history differences in every run of it, over 13,500 calls on PostgreSQL and MySQL
At least 3x calls a second per gateway core	**Met on every lookup**: 3.2-6.5x without the metadata cache, 4.0-9.4x with it, 7.8-8.1x on the same 8 cores. Paging on PostgreSQL 2.8x without the cache, 3.5x with it
p99 no worse	**Lower in every cell:** 5-20x on the same host, 2x with the database 0.5 ms away

The recommendation is go, with three conditions this run found, in What a go takes.

Contents

- Method
- Results on PostgreSQL

- Results on MySQL
- Keeping every run
- The forwarded hop, and memory
- Where the data plane doesn't help
- What a go takes
- Limits
- Run it yourself

Method

v3's, with the data plane added: the same laptop (an Intel i7-12700H on AC power, the `performance` profile and energy preference, a desk fan), the gateway on CPUs 0-7, the database on 8-11, the load generator on 12-19. Each cell is the median of 5 interleaved, rotated rounds (3 in the 64-caller block), with its min-max range in the result files. One session: PostgreSQL 2026-10-27 20:24-22:12, MySQL 22:12-00:25, the fixed-rate test 00:25-00:32.

LABEL	WHAT RUNS
v0.43.0	`c57a546`, one gunicorn worker
Python	`5844c0f` (the `feat/rust-data-plane` branch), one gunicorn worker
Python + cache	the same, `QUERYAPIGATE_METADATA_CACHE_TTL=5`
Data plane	the same, with the Rust data plane on the public port and **2** gunicorn workers behind it
Data plane + cache	the same, with the metadata cache, in Python and in the data plane

- **Gateway cores** count everything on CPUs 0-7: the data plane and every gunicorn process. A data-plane cell pays for Python's share too.
- **Runs kept** is the share of a cell's calls that left a run in the store's event log, counted by event ids, so it covers every worker. Python's builds keep 100%; a data-plane cell counts only when it does too.
- **The store** is a PostgreSQL database on the same server for PostgreSQL, and SQLite for MySQL, as in v4 and MySQL v2. Each cell starts with the event log and both server logs empty.
- **Floors.** A `/health` floor before and after every block stayed between 2,190 and 2,261 requests a second on PostgreSQL. On MySQL it read 2,177-2,253 for its first two blocks and 2,000-2,066 around the third and the start of the fourth: the multiples are measured within each round, so they hold, but MySQL's absolute figures there may be up to 10% low.
- **A first run was discarded.** On 2026-10-27's afternoon the event log wasn't emptied between cells: at the data plane's rates it grew by millions of rows a cell, slowed each later cell and filled the disk. The same run showed that most of the data plane's runs went unrecorded, which Keeping every run is about.

Results on PostgreSQL

32 callers

CALL	V0.43.0	PYTHON	PYTHON + CACHE	DATA PLANE	DATA PLANE + CACHE
Lookup, plain key	869	873	1,043	12,689 (12,622-17,597)	27,519, runs kept 51-100%
per gateway core	743	747	925	2,361: **3.2x**	3,904
Lookup, governed key	829	832	991	12,358 (12,198-12,533)	24,288, runs kept 50-100%
per gateway core	711	714	883	2,288: **3.2x**	3,685
100-row page	737	739	868	6,776 (6,745-6,872)	9,614 (9,507-9,678)
per gateway core	631	634	767	1,800: **2.8x**	2,700: **3.5x**
p99, plain lookup	44.8 ms	42.8 ms	35.9 ms	5.1 ms	2.1 ms

- **The data plane alone is 3.2x per core on a lookup.** Every one of its runs was recorded, with 2 Python workers behind it: their CPU is in the figure.
- **Flat out, the data plane + cache outran recording.** At 24,000-25,700 calls a second it kept 95-100% of its runs; at 27,000 and more, about half. The fixed-rate test below gives its figure with every run kept.
- **The page is PostgreSQL-bound:** 3.9 of its 4 cores, with the gateway at 3.6-3.8 of 8.
- **The first round after setup is fast:** 17,597 and 40,938 calls a second, against medians of 12,689 and 27,519. The run history then fills with dead rows (each version keeps 50 runs, so most rows are deleted soon after they're written), and the store's writes cost more. The medians leave that round out.

At a fixed rate

The data plane + cache, the plain lookup, 64 connections, open loop with latency correction:

OFFERED	SERVED	GATEWAY CORES	PER CORE	P50	P99	RUNS KEPT, 2 WORKERS	3 WORKERS
16,000/s	15,999	2.26	7,080	0.40 ms	1.2 ms	100%	100%
20,000/s	19,999	2.97	6,734	0.49 ms	1.8 ms	100%	100%
24,000/s	23,281	6.27	-	532 ms	910 ms	100%	100%
28,000/s	24,174	6.53	-	1.6 s	4.1 s	94%	100%

- **About 23,300 calls a second is this PostgreSQL's ceiling** (3.4 of its 4 cores), not the gateway's.
- **Below the ceiling, the gateway needs under 3 of its 8 cores for 20,000 calls a second, with every run kept:** 7.3x Python + cache per core. Run flat out against a saturated database, it spends twice the cores for the same calls, waiting busily: the closed-loop figures above understate it.

One caller, and a database 0.5 ms away

CALL	PYTHON	PYTHON + CACHE	DATA PLANE	DATA PLANE + CACHE
Plain lookup, one caller: calls/s, p50	745, 1.27 ms	970, 1.01 ms	4,785, 0.20 ms	8,347, 0.11 ms
per gateway core	930	1,111	5,748: **6.2x**	10,467: **9.4x**
PostgreSQL 0.5 ms away, one caller: calls/s, p50	217, 4.60 ms	602, 1.62 ms	475, 2.07 ms	1,317, 0.74 ms
PostgreSQL 0.5 ms away, 32 callers	885	1,031	4,956	14,234
per gateway core	746	907	4,612: **6.2x**	8,029: **8.9x**

The governed lookup is within 6% of the plain one in each cell. **Without the metadata cache, a distant store costs the data plane what it costs Python:** each call reads its three store rows across the network, one caller at a time. With the cache, one call is 0.74 ms against Python's 1.62.

Both at their best on 8 cores, 64 callers

CALL	PYTHON + CACHE, 8 WORKERS	DATA PLANE + CACHE, 2 WORKERS BEHIND IT
Plain lookup	3,537/s on 7.38 cores, p99 89.8 ms	28,020/s on 7.19 cores, p99 3.4 ms, runs kept 54%
Governed lookup	3,353/s on 7.37 cores, p99 81.1 ms	24,271/s on 6.63 cores, p99 4.7 ms, runs kept 49-97%

8.0x per core, but only partly recorded. MySQL's 64-caller block, below, is the same comparison with every run recorded.

Results on MySQL

32 callers

CALL	V0.43.0	PYTHON	PYTHON + CACHE	DATA PLANE	DATA PLANE + CACHE
Customer lookup	786	794	898	15,871	23,961
per gateway core	599	616	769	2,144: **3.5x**	3,253: **4.2x**
`order_status`	671	674	746	12,212	14,251
per gateway core	513	522	633	1,661: **3.2x**	2,515: **4.0x**
`order_status`, governed key	632	636	700	11,888	14,027
per gateway core	488	498	597	1,612: **3.2x**	2,395: **4.0x**
`stock_level`	760	770	862	4,340	4,588
per gateway core	581	597	736	2,637: **4.4x**	4,913: **6.7x**
p99, customer lookup	48.4 ms	47.0 ms	39.7 ms	3.2 ms	2.1 ms

Every run was recorded in every MySQL cell. With the cache, `order_status` and `stock_level` are MySQL-bound (3.7-3.9 of its 4 cores) while the gateway uses 0.9-5.9 of its 8; without it, the customer lookup and `order_status` use 7.4 gateway cores. The governed key, with a six-schema allow-list, a hash and a mask, costs 2-3% against the reader key.

One caller, MySQL 0.5 ms away, and 64 callers

CALL	PYTHON	PYTHON + CACHE	DATA PLANE	DATA PLANE + CACHE
Customer lookup, one caller: calls/s	865	908	5,519	7,722
per gateway core	920	973	4,395: **4.8x**	7,990: **8.2x**
`order_status`, governed, one caller	640	670	3,301	4,486
`order_status`, MySQL 0.5 ms away, one caller	342	349	1,004	1,158
`order_status`, MySQL 0.5 ms away, 32 callers	678	740	8,900	8,353

64 CALLERS, ON 8 CORES	PYTHON + CACHE, 8 WORKERS	DATA PLANE + CACHE, 2 WORKERS BEHIND IT	PER CORE
Customer lookup	2,976/s on 7.34 cores, p99 93 ms	23,619/s on 7.34 cores, p99 3.8 ms	**7.9x**
`order_status`, governed	2,252/s on 7.35 cores, p99 116 ms	13,863/s on 5.82 cores, p99 6.4 ms	**7.8x**

Each at its best, on the same 8 cores, with every run recorded, the data plane serves about 8 times as many calls a second. Python's 8 workers can't use their cores: 2,976 calls a second is 405 per core, against 769 for one worker.

Keeping every run

The data plane writes nothing to the store. It reports each run to Python, which records it: run history, the event log, the live feed, `/metrics` (ADR 0028, section 3, item 8). The first full run showed Python couldn't keep up:

	BEFORE	NOW
Python's work per reported run	27 µs	6.9 µs
Runs one Python process records a second	about 3,000	about 17,000
Runs lost without being counted	most of them, above 8,600 a second	none: counted as `dropped`
Data plane + cache, flat out, with every run recorded	-	about 24,000 a second, 2 workers

What changed:

- **The data plane sends each run already serialized:** its history entry as `json.dumps` writes it, and its labels and timings. Python queues a batch's history rows and events in one go, and counts each set of labels once.
- **History inserts only what it keeps.** Each version keeps its newest 50 runs, so of a batch of 500 it now writes 50, not 500 to delete 450. Events go as one multi-row insert.
- **Runs the data plane can't queue are counted** in `queryapigate_history_runs_total{outcome="dropped"}`, with the next batch.
- **Two Python workers behind it, not one:** one process's writer keeps up with about 17,000 runs a second.

The forwarded hop, and memory

The two figures ADR 0028 also asked for, measured on 2026-10-28, 07:41-07:57: PostgreSQL, the metadata cache on in both builds, each latency the middle of 3 runs of 30 s.

	PYTHON + CACHE, DIRECT	THROUGH THE DATA PLANE
Write (forwarded), one caller: p50 / p99	1.43 / 2.28 ms	1.78 / 3.52 ms
Write (forwarded), 400 a second: p50 / p99	1.77 / 5.04 ms	2.12 / 4.40 ms
Lookup (served), one caller: p50 / p99	1.38 / 3.04 ms	0.18 / 0.64 ms
Lookup (served), 400 a second: p50 / p99	1.58 / 4.71 ms	0.63 / 1.38 ms
Memory (VmRSS) after the cells	master 48 MB, worker 52 MB	data plane 19 MB; master 48 MB, 2 workers 64 MB each

- **A forwarded call pays 0.35 ms at p50,** at both loads. At 400 a second its p99 is no worse.
- **The data plane holds 19 MB.** Each Python worker behind it holds 12 MB more than a worker on its own.
- One caller's lookup through Python + cache took 1.38 ms here, against 1.01 ms in the decision run's 1-caller block: this was a separate session, so compare within the table, not across runs.

Where the data plane doesn't help

- **Writes** are forwarded to Python: 1,653 calls a second through the data plane with its 2 workers, against 1,030 for one worker on its own: **0.8x per core,** and 0.35 ms more at p50 (above). The forwarding hop costs CPU, and writes were never the data plane's to serve.
- **A 100-row page on PostgreSQL without the cache: 2.8x per core,** below the bar. PostgreSQL sets the limit there (3.9 of 4 cores); with the cache, 3.5x.
- **The first call to each query, key and schema, and every call using a feature the data plane leaves to Python** (ADR 0028, section 4), cost what Python costs, plus the hop. The figures here are steady state: the same few queries, called again and again.

What a go takes

1. **At least 2 Python workers behind the data plane.** With 1, about half its runs went unrecorded. 2 kept every run to about 24,000 calls a second here; above that, runs are dropped and counted.
2. **A store built for the run volume.** Every run recorded at 20,000 a second is 1.7 billion a day. The event log keeps each for at least an hour, whatever the count; SQLite and a PostgreSQL store can't hold that for long. A store for runs at this rate is ADR 0029's subject.
3. **The pull request it promises:** the CI job that builds the data plane and runs the differential gate, cargo's Dependabot ecosystem, the threat-model entries, and its documentation.

Limits

v3's: one machine, a laptop's power limit, a simulated network (one-way delay, no jitter or loss), cheap queries on data that fits in memory, and no soak test. Also:

- **Steady state only:** a few queries, called again and again. A deployment with many distinct queries and keys spends more of its calls on Python's plan-giving first call.
- **The recording ceiling is this machine's:** about 24,000 runs a second with 2 workers and a local store.
- **The run history fills with dead rows** under the data plane's write rate, and later cells pay for it: see the first-round outliers. A run longer than this one's 30 s cells would show how far that goes.

Run it yourself

```shell
pip install -e ".[server,postgres,mysql]"
cargo build --release --manifest-path dataplane/Cargo.toml
python benchmarks/oltp.py --out /tmp/qag-dp-pg --base v0.43.0 --head feat/rust-data-plane \
    setup env d1 d2 d3 d4 env teardown                                                  # 1 h 50
min
python benchmarks/tpcc_mysql.py --out /tmp/qag-dp-mysql --go-tpc /path/to/go-tpc --refs
v0.43.0,feat/rust-data-plane \
    setup env d1 d2 d3 d4 env teardown                                                  # 2 h 15
min
python benchmarks/oltp_report.py /tmp/qag-dp-pg/results.jsonl
python benchmarks/tpcc_mysql_report.py /tmp/qag-dp-mysql/results.jsonl
```

Pass `--out` as an absolute path, and benchmark committed code: each build is a `git archive` of its ref. The 2026-10-27 runs are in `benchmarks/results/dataplane-postgres-2026-10-27/` (with `fixed-rate.jsonl`) and `benchmarks/results/dataplane-mysql-2026-10-27/`.